SOFTWARE ENGINEERING PROJECT

Sonar Retina

BY

Thanawas Sirilertsathit
Pattharamon Dumrongkittikule

DEPARTMENT OF COMPUTER ENGINEERING
FACULTY OF ENGINEERING
KASETSART UNIVERSITY

Academic Year YYYY

Sonar Retina

BY

Thanawas Sirilertsathit
Pattharamon Dumrongkittikule

A Project Submitted in Partial Fulfillment of the Requirement for the Degree of Bachelor of Engineering (Software and Knowledge Engineering)
Department of Computer Engineering
Faculty of Engineering
KASERTSART UNIVERSITY
Academic Year YYYY

Approved By:

Advisor . . . . . **Date** . . . . . .
(Project Advisor's Name)

Co-Advisor . . . . . **Date** . . . . . .
()

Head of Department . . . . . **Date** . . . . . .
(Head of Department's Name)

Abstract

For deaf, the impact of hearing loss is the disconnection from social environment. This project introduces a mobile application that uses AI to restore the missing ability by visualizing sound in spatial context. Rather than focusing on emergencies and safety, this app identifies and locates sound in daily life such as bird sound and baby crying. By converting those sounds into 2D visual information, the application serves as an assessibility tool that transforms sound into a visible location and restoring a sense of hearing.

Acknowledgement

Put your acknowledgement paragraph here.

Thanawas Sirilertsathit
Pattharamon Dumrongkittikule

Table of Contents

List of Tables

List of Figures

Chapter 1

Introduction

1.1 Background

<TIP: A background chapter in a book serves the purpose of providing essential context, information, or history that is relevant to the overall understanding of the book's subject matter. This chapter typically appears early in the book and aims to set the stage for the reader by offering background details that are crucial for comprehending the main narrative. />

1.2 Problem Statement

<TIP: A problem statement refers to a clear central issue, challenge, or question that the project aims to address or explore. It is a declaration that highlights the specific problem the author intends to examine, discuss, or solve throughout the course of the project. />

1.3 Solution Overview

<TIP: A software solution overview provides a high-level and concise description of a software product or system. It serves as an introduction to the software, offering a glimpse into its key features, functionalities, and the problems it aims to address. This overview is often presented in documentation, marketing materials, or other communication channels to give stakeholders, potential users, or decision-makers a quick understanding of what the software does and why it is valuable. />

1.3.1 Features

1. Feature Name: Short Description of Feature
2. Feature Name: Short Description of Feature

1.4 Target User

<TIP: The target user in a software project refers to the specific group or demographic of individuals for whom the software is designed and developed. Identifying the target user is a crucial step in the software development process as it helps the development team tailor the software to meet the needs, preferences, and requirements of that particular user group. Understanding the characteristics, behaviors, and expectations of the target users is essential for creating a user-friendly and effective software solution.

Here are some key aspects related to defining the target user in a software project:

Demographics: This includes factors such as age, gender, occupation, education level, and other demographic characteristics. Different age groups or professional backgrounds may have distinct preferences and requirements when it comes to software usability.

Skill Level: Consideration of the users' technical proficiency and familiarity with similar software or technology. The level of technical expertise can influence the complexity of the user interface, the need for tutorials or documentation, and other user support features.

Industry or Domain: For software solutions designed for specific industries or domains, understanding the unique challenges, workflows, and terminology within that industry is crucial. Tailoring the software to meet industry-specific needs is often necessary. />

1.5 Terminology

<TIP: Terminology refers to the specific language, jargon, or specialized vocabulary used to describe concepts, ideas, or subjects within a

particular field or domain. The use of terminology is often essential for clarity and precision, especially in books that cover technical, scientific, academic, or specialized topics. />

Chapter 2

Literature Review and Related Work

2.1 Competitor Analysis

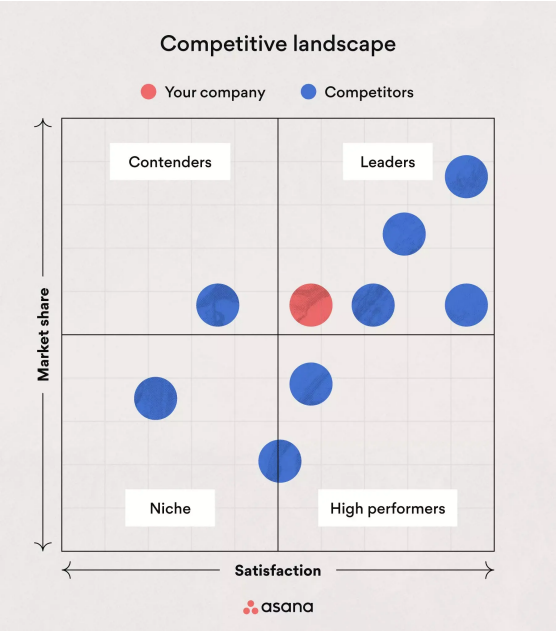

Figure 2.1: Competitive Landscape by Asana

<TIP: Refer to an article "How to create a competitive analysis (with examples)" by Asana. You can use the Competitor Landscape or Competitor Analysis Framework for your project. />

2.2 Literature Review

<TIP: Review academic papers or technical reports that are relevant to your project. For each source, briefly summarize its key findings and

then explain how it informs your own design or implementation decisions. The goal is not just to describe what others have done, but to build a justification for your own technical choices. For example, if you chose a particular algorithm or architecture, cite the work that supports that decision. />

Chapter 3

Requirement Analysis

3.1 Stakeholder Analysis

<TIP: List your stakeholders for your project here/>

Stakeholders are individuals, groups, or entities that have an interest, concern, or stake in a particular project, decision, organization, or system. These are individuals or groups who can affect or be affected by the outcomes of your project.

3.2 User Stories and Use Cases

<TIP: For each stakeholder, write user stories that capture what they need from the system: "As a [stakeholder], I want to [action] so that [benefit]." For your most important user stories, expand them into use cases. A use case is a detailed description of how a system interacts with an external entity (such as a user or another system) to accomplish a specific goal. Use cases provide a high-level view of the functionality of a system and help in capturing and documenting its requirements from the perspective of end users.

Make sure to use the appropriate type of use case for each scenario (brief, casual, and fully-dressed use case). />

3.3 Use Case Diagrams

<TIP: Include a UML Use Case Diagram that shows all actors and their relationships to the use cases you described. Also provide the written description for each use case. />

3.4 User Interface Design

<TIP: Put the initial design of your application here. You can showcase a detailed design of a specific page or a sitemap of your application. See an example below./>

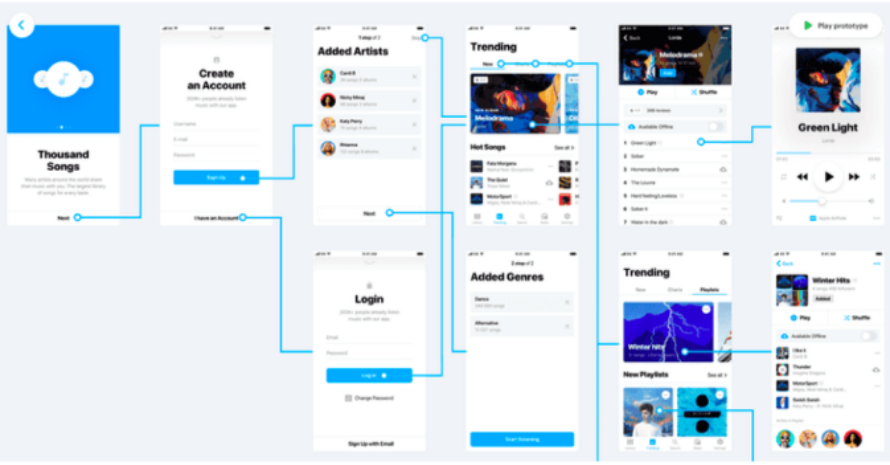

Figure 3.1: User Interface Design

Chapter 4

Software Architecture Design

4.1 System Architecture Overview

<TIP: Provide a high-level description of your system's structure. Include either a Domain Model (to describe core business concepts and their relationships) or a Design Class Diagram (to describe how your code is organized into classes and packages), whichever best communicates your design. Accompany your diagram with a short paragraph explaining the key components and how they interact. />

4.2 Software Design

<TIP: Describe the runtime behavior of your system. Include a Sequence Diagram for your most important use cases. Focus on scenarios with non-trivial interactions between components. If your project involves a novel algorithm, describe it here and explain why you designed it this way. />

4.3 AI and Data Design

4.3.1 Data Sources and Collection

<TIP: Describe where your data comes from. Is it collected from users, scraped from the web, sourced from a public dataset, or provided by an external API? Explain how much data you have or plan to collect, and any challenges you anticipate in obtaining it. Explain how data is cleaned, transformed, or prepared before use. />

4.3.2 Model Design

<TIP: Describe the AI model or external AI service you plan to use. Explain why you chose it over alternatives. This can connect back to your competitor or literature review in Chapter 2. If you are training your own model, describe the architecture and training approach. If you are using a third-party model or API, describe how it is integrated into your system and any prompt design or fine-tuning involved. />

4.3.3 Evaluation

<TIP: Describe the metrics you plan to use to evaluate your AI model. />

Chapter 5

Software Development

5.1 Software Development Methodology

<TIP: Describe your software development methodology in this section. />

5.2 Technology Stack

<TIP: Describe your technology stack here. See the following example from ThaiProgrammer.org />

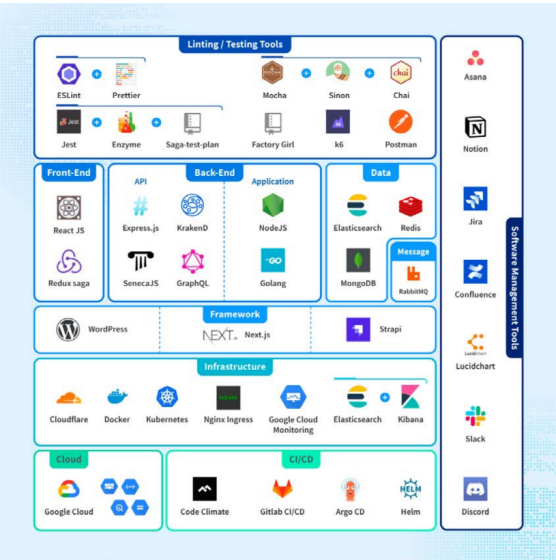

Figure 5.1: Example technology stack

5.3 Coding Standards

<TIP: Describe your coding standard for this project here. />

5.4 Schedule

<TIP: Show the schedule of your project. Use a Gantt chart. The items should agree with features you listed in Chapter 1. />

5.5 Progress Tracking Report

<TIP: Show that you have been working on this project overtime. It can be in the form of a burndown chart or a contribution graph from GitHub./>

Chapter 6

Deliverables

6.1 Software Solution

<TIP: Share a link to your Github repository. Showcase screenshots of the application and briefly describe each page here. />

6.2 Test Report

<TIP: Describe how you test your project. Place a test report here. If you use continuous integration and deployment (CI/CD) tools, describe your CI/CD method here. />

Chapter 7

Conclusion and Discussion

7.1 Summary of Achievements

<TIP: Summarize what your project accomplished. Refer back to your problem statement from Chapter 1 and explain the degree to which your solution addressed it. Highlight the most significant features or results delivered. />

7.2 Known Limitations

<TIP: Describe the current limitations of your solution honestly. These may include technical constraints (e.g., scalability, performance), scope limitations (features not yet implemented), data limitations (e.g., small training sets, biased data), or usability issues discovered during testing. />

7.3 Future Work

<TIP: Propose concrete improvements or extensions that could be made to this project given more time or resources. These should follow logically from your limitations. For example, if your model was trained on limited data, future work might include collecting a larger dataset. Avoid vague suggestions like "improve the UI"; instead, be specific about what would be changed and why. />

Reference

Bibliography

[1] Overleaf, “Learn latex in 30 minutes,” https://www.overleaf.com/learn/latex/Learn_LaTeX_in_30_minutes.

Appendix A

Appendix A: Example

<TIP: Put additional or supplementary information/data/figures in appendices. />

Appendix B

Appendix B: About LaTeX

LaTeX (stylized as LaTeX) is a software system for typesetting documents. LaTeX markup describes the content and layout of the document, as opposed to the formatted text found in WYSIWYG word processors like Google Docs, LibreOffice Writer, and Microsoft Word. The writer uses markup tagging conventions to define the general structure of a document, to stylize text throughout a document (such as bold and italics), and to add citations and cross-references.

LaTeX is widely used in academia for the communication and publication of scientific documents and technical note-taking in many fields, owing partially to its support for complex mathematical notation. It also has a prominent role in the preparation and publication of books and articles that contain complex multilingual materials, such as Arabic and Greek.

Overleaf has also provided a 30-minute guide on how you can get started on using LaTeX. [1]